

Testing Topological Memory

Storage capacity, retrieval accuracy, I/O performance

This test measures the memory retrieval accuracy as a function of the memory density.

Process: In each iteration, a new batch of random associations $A \rightarrow B$ is stored in memory.

We then retrieve the same batch with inputs A and compare the outputs with B , measuring overlaps and Hamming distances. Optionally, random noise may be added to inputs A , testing the retrieval accuracy in the presence of additive or subtractive noise.

We use this setup to verify that different memory backends (C, Mathematica, Python, ...) behave exactly the same.

Note that the native Mathematica implementation of topological associative memory is much slower than the performance-optimized C backend.

Requirements

```
<< "Memory.m"
```

Configuration

```
config = <| "A_parameters" → {1000, 10}, "B_parameters" → {1001, 10} |>;
```

```
batchsize = 50000;  
iterations = 40;  
randomnoise = 0; (* Additive (positive) or subtractive (negative) noise. *)
```

Test setup

```
RandomSet[{n_Integer, p_Integer}] := Sort[RandomSample[Range[n], p]]
```

```
RandomAssociation[config_Association] :=  
  {RandomSet[config["A_parameters"]], RandomSet[config["B_parameters"]]}
```

```
AddNoise[ A : {__Integer}, {n_Integer, p_Integer}, noise_Integer] :=  
  Which[  
    noise == 0, A,  
    noise > 0, Union[A, RandomInteger[{1, n}, noise]],  
    noise < 0, Sort[RandomSample[A, Length[A] + noise]]  
  ]
```

```
Overlap[ A : {__Integer}, B : {__Integer} ] :=  
  Length[Intersection[A, B]]
```

```
Distance[ A : {__Integer}, B : {__Integer} ] :=  
  Length[Complement[A, B]] + Length[Complement[B, A]]
```

```

Test[mem_Association, iter_Integer, datasize_Integer, noise_Integer] :=
  Module[ {data, inp, out, storetime, retrievetime, distance, overlap, memsize},

    memsize = mem["B_parameters"][[1]] *
      mem["A_parameters"][[1]] (mem["A_parameters"][[1]] - 1) / 2;

    data = Table[ RandomAssociation[mem], batchsize];

    storetime = First[AbsoluteTiming[ mem["store"] @@@ data]];

    inp = AddNoise[# , mem["A_parameters"], randomnoise] & /@ data[[All, 1]];

    retrievetime = First[AbsoluteTiming[ out = mem["retrieve"] /@ inp ]];

    distance = Total[Table[ Distance[out[[i]], data[[i,2]], {i, 1, batchsize}]];

    overlap = Total[Table[ Overlap[out[[i]], data[[i,2]], {i, 1, batchsize}]];

    Print[ "|",
      PaddedForm[ iter*batchsize, 10], " |",
      PaddedForm[ Round[batchsize/storetime], 10], " |",
      StringPadLeft[ToString[DecimalForm[ N[mem["memorycount"]]/memsize, 6],
      PaddedForm[ Round[ batchsize/retrievetime], 10], " |",
      PaddedForm[ N[distance/batchsize], {9,6}], " |",
      PaddedForm[ N[overlap/batchsize], {9,6}] , " |" ];

  ]

```

Run test

```
memory = Memory[config];
```

```

Print[ "Configuration: ", memory]
Print[ "Additive noise: ", randomnoise];
Print[];

Print["|      items |   writes/s |   density |   reads/s |   distance |   c
Print["|-----|-----|-----|-----|-----|-----|-----|

```

```

Do[ Test[memory, i, batchsize, randomnoise], {i, 1, iterations}]

```

Configuration: $\langle |A_parameters \rightarrow \{1000, 10\}, B_parameters \rightarrow \{1001, 10\}, T \rightarrow 7,$
 $store \rightarrow store\$1577380, retrieve \rightarrow retrieve\$1577380, clear \rightarrow clear\$1577380,$
 $memorycount \rightarrow memorycount\$1577380, backend \rightarrow Mathematica | \rangle$

Additive noise: 0

items	writes/s	density	reads/s	distance	overlap
50000	11616	0.044002	9272	0.000000	10.000000
100000	11544	0.086070	8906	0.000000	10.000000
150000	10808	0.126279	8335	0.000000	10.000000
200000	10788	0.164729	8086	0.000000	10.000000
250000	11217	0.201479	8352	0.000000	10.000000
300000	9964	0.236615	8146	0.000000	10.000000
350000	10490	0.270205	8475	0.000000	10.000000
400000	10655	0.302312	8538	0.000000	10.000000
450000	10477	0.333011	8318	0.000000	10.000000
500000	10375	0.362364	8436	0.000000	10.000000
550000	10479	0.390419	8529	0.000000	10.000000
600000	10457	0.417250	8518	0.000000	10.000000
650000	10492	0.442899	8522	0.000000	10.000000
700000	10460	0.467413	8508	0.000000	10.000000
750000	10500	0.490849	8449	0.000000	10.000000
800000	10358	0.513256	8425	0.000000	10.000000
850000	10588	0.534680	8501	0.000000	10.000000
900000	10473	0.555160	7886	0.000000	10.000000
950000	10186	0.574726	8393	0.000000	10.000000
1000000	9884	0.593442	8174	0.000000	10.000000
1050000	10498	0.611331	8436	0.000000	10.000000
1100000	10337	0.628437	8510	0.000000	10.000000
1150000	10633	0.644797	8503	0.000000	10.000000
1200000	10655	0.660429	8514	0.000020	10.000000
1250000	10600	0.675367	8449	0.000040	10.000000
1300000	10564	0.689657	8474	0.000100	10.000000
1350000	10551	0.703310	8516	0.000320	10.000000
1400000	10444	0.716372	8511	0.000500	10.000000

	1450000		10515		0.728858		8552		0.001040		10.000000	
	1500000		10499		0.740795		8547		0.002300		10.000000	
	1550000		10440		0.752200		8512		0.004000		10.000000	
	1600000		10470		0.763110		8556		0.008280		10.000000	
	1650000		10482		0.773536		8567		0.015140		10.000000	
	1700000		10451		0.783503		8586		0.024180		10.000000	
	1750000		10506		0.793035		8559		0.039240		10.000000	
	1800000		10088		0.802140		8460		0.066700		10.000000	
	1850000		10535		0.810845		8466		0.102980		10.000000	
	1900000		10616		0.819172		8502		0.160340		10.000000	
	1950000		10600		0.827125		8447		0.245940		10.000000	
	2000000		10532		0.834727		8468		0.354100		10.000000	